Lab 2 Part 2 — LLM Compression of Qwen3-4B

Student ID: 314551011

0. Setup and how the numbers are measured

- **Model:** `Qwen/Qwen3-4B`, loaded in fp16. Every result comes from the provided `run_eval.py` / `eval_ppl_wikitext2` (WikiText-2 test, 2048-token windows). The evaluation code is unchanged.
- **Target layers:** all `nn.Linear` layers inside the 36 decoder blocks (`q/k/v/o_proj`, `gate/up/down_proj`). That is 3.633 B parameters; the original size is 16 bits/parameter. The embedding / tied `lm_head` is not compressed and not counted.
- **Compressed size:**
  - Weights are fake-quantized: they are de-quantized back to fp16 for the forward pass.
  - The size is counted as a packed format would store it. That is *b* bits per integer weight, plus a 16-bit fp16 scale and a *b*-bit zero point for every quantization group, plus every SVD factor.
  - `common.record_base()` records the original size, and each compression step records the bits of the layer it rewrote.
  - `compression_rate()` in `evaluate.py` returns `1 − Σ bits / (16 · base_numel)`.
- **Calibration:** 128 random 2048-token windows from the WikiText-2 **train** split (seed 0). The test split is never seen during compression.
- **Fine-tuning:** none. Every weight change is a closed-form, layer-local update: GPTQ error feedback, a whitened SVD, or a least-squares refit.
- **Pipeline (`compress_lib.py`):**
  - The model is processed one decoder layer at a time ("true sequential"). Within a layer the order is [q,k,v] → [o] → [gate,up] → [down].
  - Each group's input Hessian $H = 2XX^T$ is collected from the *already compressed* preceding layers. Errors therefore do not compound blindly.
- **Base model PPL: 13.638.**

Building blocks used in the four cases:

Technique	What it does	Extra storage counted
GPTQ (act-order, asymmetric)	Column-by-column rounding with Hessian error feedback. Groups are fitted on the original column order (static groups), so no permutation index is stored.	fp16 scale + *b*-bit zero per group
Whitened SVD (SVD-LLM)	$W \approx A \cdot B$. The SVD is taken of $W \cdot S$, where $S = \text{chol}(XX^T)$, so truncation minimizes the output error $\|(W-AB)X\|$.	Both factors, $k \cdot (m+n)$ values
Structured MLP pruning	Removes whole intermediate channels: rows of gate/up_proj and columns of down_proj. Channels are ranked by the OBS saliency $\|W_{down}[:,j]\|^2 / [H^{-1}]_{jj}$, and down_proj is refit on the kept channels by least squares, $W' = W \cdot H[:,K] \cdot H[K,K]^{-1}$.	None: dense, smaller matrices

1. Exploration

All runs use the same code path as the final cases (H200, one GPU, about 4 minutes of compression per run).

Uniform GPTQ:

Config	bits/weight	CR	PPL
4-bit, group 128	4.156	74.02%	14.30
4-bit, per-row	4.006	74.96%	14.23
4-bit, per-row + MSE clip	4.006	74.96%	14.02
3-bit, group 128	3.148	80.32%	15.33
3-bit, group 512	3.037	81.02%	16.73
3-bit, group 640	3.030	81.06%	17.25

3-bit, per-row	3.006	81.21%	19.18
3-bit, per-row + MSE clip	3.006	81.21%	22.38

Sensitivity of each Linear type: that projection type, in all layers, at 3-bit per-row; everything else fp16.

Layer type	share of params	PPL	ΔPPL	ΔPPL per 1% of params
q_proj	10.4%	13.81	+0.17	0.017
k_proj	2.6%	14.46	+0.82	**0.315**
v_proj	2.6%	13.92	+0.28	0.108
o_proj	10.4%	13.82	+0.18	0.017
gate_proj	24.7%	14.20	+0.56	0.023
up_proj	24.7%	14.22	+0.58	0.023
down_proj	24.7%	15.17	**+1.53**	0.062

Sensitivity by depth: 9 consecutive layers at 3-bit per-row; everything else fp16.

Layers	0–8	9–17	18–26	27–35
PPL	14.46	**14.20**	14.38	16.54

2. Case 1 — CR > 75%, PPL < 15

Strategy.

- Mixed-precision GPTQ, act-order, asymmetric.
- All target layers are 4-bit with one fp16 scale and a 4-bit zero per output row (4.006 bits/weight).
- `down_proj` of layers 16–19 is 3-bit with group size 128.

Result. CR 75.11 %, PPL 14.26.

Analysis.

- Pure 4-bit cannot pass 75 %. Any scale or zero overhead pushes the average above 4 bits: per-row reaches 74.96 %, group 128 only 74.02 %.
- A few percent of the weights therefore have to drop to 3 bits. Section 1 shows where this is cheapest:
    - The middle layers (9–17 / 18–26) are the least sensitive.
    - `down_proj` is large, so a few layers give the needed 0.15 % CR.
    - Group-128 scales on those few matrices keep their error small.
- The alternative of putting `k_proj`/`v_proj` in every layer at 3-bit g128 reached 75.24 % CR but PPL 14.95. That is right at the limit, matching the k_proj sensitivity measured above.
- Per-row 4-bit was slightly *better* than group-128 here (14.23 vs 14.30). With act-order GPTQ, error feedback compensates most of the coarse-scale error at 4 bits.

3. Case 2 — CR ≥ 81%, PPL < 20

Strategy.

- 3-bit GPTQ (act-order, asymmetric) on every target layer.
- Group size 512: one fp16 scale + 3-bit zero per 512 weights of a row, i.e. 3 + 19/512 = 3.037 bits/weight.

Result. CR 81.02 %, PPL 16.73.

Analysis.

- ≥ 81 % allows at most 3.04 bits/weight, so the only free choice is how to spend the ~0.04 bits beyond the integer codes. Finer scales help a lot at 3 bits:
    - per-row (0.006 bits overhead): 19.18
    - group 640: 17.25
    - group 512: 16.73

    - group 128: 15.33, but that costs 0.15 bits (80.3 %) and exceeds the budget.

- Group 512 is the finest grouping that still fits the budget. It gains 2.4 PPL over the per-row configuration for 0.03 bits.
- MSE-optimal clipping (Part 1 Q8) *hurt* at 3 bits (19.18 → 22.38) but helped at 4 bits (14.23 → 14.02):
    - Clipping lowers the rounding error of a row in isolation.
    - GPTQ then pushes the accumulated error of previous columns into later columns. With a clipped range, those updated values saturate more often, and clamping error is not compensated.

4. Case 3 — SVD CR ≥ 20%, overall CR = 50%, PPL < 50

Strategy.

1. **Whitened SVD on every target layer.**
    - Rank $k = \lfloor 0.8 \cdot m \cdot n/(m+n) \rfloor$, so each matrix keeps 80 % of its parameters as factors A (m×k) and B (k×n). The singular values are split as $\sqrt{\Sigma}$ between A and B.
    - **SVD compression ratio: 20.02 %.**
2. **INT8 GPTQ on part of the factors.**
    - Quantized: the factors of all MLP projections and of `o_proj` in layers 0–5 (8-bit, per-row scale + zero). B uses the input Hessian H; A uses BHB^T, the Hessian of its own input.
    - The remaining attention factors stay fp16.
    - This lands the overall ratio at 50 %; INT8 on every factor would give ≈ 59.6 %.

Result. CR 50.27 % (SVD 20.02 %), PPL 19.87.

Analysis.

- Almost all of the degradation comes from the rank truncation: SVD alone gives 19.90, SVD + INT8 gives 19.87. INT8 of the factors is lossless within noise.
- Whitening is essential. Truncating the plain SVD of W ignores that the input channels of LLM activations have very different magnitudes (outlier channels).
- 20 % parameter reduction is already a large rank cut for these near-square matrices: q_proj keeps rank 1260 / 2560, and the MLP matrices keep 1621 / 2560.
- Low-rank structure is therefore a far less efficient use of bits than quantization:
    - SVD at 20 % CR: PPL 19.9.
    - 3-bit GPTQ at 81 % CR: PPL 16.7.
- "Overall = 50 %" forces most of the remaining budget to be spent on keeping the factors at high precision.

5. Case 4 — Pruning rate ≥ 20%, overall CR ≥ 85%, PPL < 50

Strategy.

1. **Structured MLP pruning.**
    - In every layer, 29 % of the 9728 intermediate channels are removed (2821 → 6907 kept), using OBS saliency and the least-squares refit of `down_proj`.
    - **Pruning rate: 21.47 % of all target parameters.**
2. **3-bit GPTQ on all remaining weights.**
    - Group size 640, i.e. 3.03 bits per kept weight.
    - Pruning happens inside the sequential pass, right before gate/up are quantized. The pruned MLP is then quantized with Hessians that already reflect the pruning.

Result. CR 85.13 % (pruning 21.47 %), PPL 20.33.

Analysis.

- The pruning is structured so that pruning really reduces storage. An unstructured 20 % mask costs ≥ 0.72 bits/weight (its entropy) or 1 bit as a bitmap, which would leave < 2 bits for the values. Removing whole

channels leaves dense, smaller matrices with zero index overhead.
- Pruning alone (27.5 % of channels) gives PPL 16.29. With 3-bit on top, the errors roughly add.
- Pruning *slightly more* channels (29 % instead of 28 %) frees enough bits to switch from per-row to group-640 scales. That improved PPL from 22.73 to 20.33, consistent with Case 2: at 3 bits, scale granularity matters more than a few extra channels.
- The least-squares refit of `down_proj` is what makes a 29 % channel cut viable without fine-tuning. It redistributes the removed channels' contribution onto the correlated kept channels.

6. Conclusion

Case	Method	CR	PPL	Limit
1	4-bit GPTQ + 3-bit g128 down_proj (4 mid layers)	75.11 %	14.26	< 15
2	3-bit GPTQ, g512	81.02 %	16.73	< 20
3	whitened SVD 20 % + INT8 factors	50.27 %	19.87	< 50
4	structured MLP pruning 21.5 % + 3-bit GPTQ g640	85.13 %	20.33	< 50

- **Trade-off.**
  - PPL grows slowly down to ~4 bits (13.64 → 14.2) and then sharply: 3-bit costs +3 to +5.5 PPL depending on the scale granularity.
  - At a fixed budget, *how* bits are spent matters as much as how many. Case 2 at 81 % CR (16.7) beats Case 3 at only 50 % CR (19.9).
- **Sensitive layers.**
  - `down_proj` is the most damaging to quantize in absolute terms. Its input, after the SwiGLU gate, is heavy-tailed.
  - `k_proj` is the most sensitive per parameter. Errors in keys perturb every attention score, and Qwen3 has only 8 KV heads.
  - The last quarter of the network (layers 27–35) is far more sensitive than the middle. Low precision belongs in the middle layers.
- **Quantization vs. SVD vs. pruning.**
  - For the same storage, quantization with Hessian-aware rounding (GPTQ) is by far the most effective. SVD and pruning remove information irreversibly, while 3–4-bit codes keep every weight approximately.
  - SVD and pruning are useful only on top of quantization, and only with activation awareness: the whitened SVD, and OBS scoring + least-squares compensation.
- **Limitations.**
  - Mixed-precision placement was chosen from a coarse sensitivity sweep, not a search. Layer-wise bit allocation, e.g. 4-bit for late layers and k_proj with 3-bit elsewhere, would likely improve Cases 2 and 4.
  - Calibration on WikiText-2 train is in-domain for the evaluation; a generic corpus (C4) would give slightly higher PPL.
  - The weights are fake-quantized. The stated sizes assume a packed kernel, which was not implemented.
- **Most effective strategy:** GPTQ with act-order and the finest quantization groups the bit budget allows. Any extra reduction needed (pruning, low rank) should be placed in the middle layers and avoid `k_proj`/`down_proj` of the last layers.

7. Reproduction

```
cd Part2
bash 314551011_run.sh     # pip install -r requirements.txt, then cases 1-4 via run_eval.py
```

- Each case is also `python run_eval.py --case N --rate R --wbits B`. The case definitions live in

`case1.py`–`case4.py`.

- About 17 minutes total on one H200 or H100: about 4 minutes of compression per case plus evaluation.
- Tested in a fresh Python 3.11 / torch 2.5.1+cu121 environment.